The difference between the two execution plans is clearly visible in the "Rows" column. According to the optimizer's estimate, the query ultimately matches 17 records. The index scan in the first execution plan delivers 333 rows nevertheless. The database must then load these 333 rows from the table to apply the `LIKE` filter which reduces the result to 17 rows. In the second execution plan, the index access does not deliver those rows in the first place so the database needs to execute the `TABLE ACCESS BY INDEX ROWID` operation only 17 times.

You should also note that the cost value of the `INDEX RANGE SCAN` operation grew from two to three because the additional column makes the index bigger. In view of the performance gain, it is an acceptable compromise.



Warning

Don't introduce a new index for the sole purpose of filter predicates. Extend an existing index instead and keep the maintenance effort low. With some databases you can even add columns to the index for the primary key that are not part of the primary key.

This trivial example seems to confirm the common wisdom to index every column from the **`where`** clause. This "wisdom", however, ignores the relevance of the column order which determines what conditions can be used as access predicates and thus has a huge impact on performance. The decision about column order should therefore never be left to chance.

The index size grows with the number of columns as well—especially when adding text columns. Of course the performance does not get better for a bigger index even though the logarithmic scalability limits the impact considerably. You should by no means add all columns that are mentioned in the **`where`** clause to an index but instead only use index filter predicates intentionally to reduce the data volume during an earlier execution step.

Index-Only Scan

The index-only scan is one of the most powerful tuning methods of all. It not only avoids accessing the table to evaluate the **where** clause, but avoids accessing the table completely if the database can find the selected columns in the index itself.

To cover an entire query, an index must contain *all* columns from the SQL statement—in particular also the columns from the **select** clause as shown in the following example:

```
CREATE INDEX sales_sub_eur
    ON sales
     ( subsidiary_id, eur_value );

SELECT SUM(eur_value)
  FROM sales
 WHERE subsidiary_id = ?;
```

Of course indexing the **where** clause takes precedence over the other clauses. The column SUBSIDIARY_ID is therefore in the first position so it qualifies as an access predicate.

The execution plan shows the index scan without a subsequent table access (TABLE ACCESS BY INDEX ROWID).

```
-------------------------------------------------------------
| Id  | Operation         | Name          |  Rows | Cost |
-------------------------------------------------------------
|   0 | SELECT STATEMENT  |               |     1 |  104 |
|   1 |  SORT AGGREGATE   |               |     1 |      |
|*  2 |   INDEX RANGE SCAN| SALES_SUB_EUR | 40388 |  104 |
-------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   2 - access("SUBSIDIARY_ID"=TO_NUMBER(:A))
```

The index covers the entire query so it is also called a *covering index*.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>



Note

If an index prevents a table access it is also called a *covering index*.

The term is misleading, however, because it sounds like an index property. The phrase index-only scan correctly suggests that it is an execution plan operation.

The index has a copy of the EUR_VALUE column so the database can use the value stored in the index. Accessing the table is not required because the index has all of the information to satisfy the query.

An index-only scan can improve performance enormously. Just look at the row count estimate in the execution plan: the optimizer expects to aggregate more than 40,000 rows. That means that the index-only scan prevents 40,000 table fetches—if each row is in a different table block. If the index has a good clustering factor—that is, if the respective rows are well clustered in a few table blocks—the advantage may be significantly lower.

Besides the clustering factor, the number of selected rows limits the potential performance gain of an index-only scan. If you select a single row, for example, you can only save a single table access. Considering that the tree traversal needs to fetch a few blocks as well, the saved table access might become negligible.



Important

The performance advantage of an index-only scans depends on the number of accessed rows and the index clustering factor.

The index-only scan is an aggressive indexing strategy. Do not design an index for an index-only scan on suspicion only because it unnecessarily uses memory and increases the maintenance effort needed for **`update`** statements. See Chapter 8, "*Modifying Data*". In practice, you should first index without considering the **`select`** clause and only extend the index if needed.

Index-only scans can also cause unpleasant surprises, for example if we limit the query to recent sales:

```
SELECT SUM(eur_value)
  FROM sales
 WHERE subsidiary_id = ?
   AND sale_date > ?;
```

Without looking at the execution plan, one could expect the query to run faster because it selects fewer rows. The **where** clause, however, refers to a column that is not in the index so that the database must access the table to load this column.

```
-------------------------------------------------------------------
|Id | Operation                    | Name       | Rows  |Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT             |            |     1 | 371 |
| 1 |  SORT AGGREGATE              |            |     1 |     |
|*2 |   TABLE ACCESS BY INDEX ROWID| SALES      |  2019 | 371 |
|*3 |    INDEX RANGE SCAN          | SALES_DATE| 10541 |  30 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   2 - filter("SUBSIDIARY_ID"=TO_NUMBER(:A))
   3 - access("SALE_DATE">:B)
```

The table access increases the response time although the query selects fewer rows. The relevant factor is not how many rows the query delivers but how many rows the database must inspect to find them.



Warning

Extending the **where** clause can cause "illogical" performance behavior. Check the execution plan before extending queries.

If an index can no longer be used for an index-only scan, the optimizer will choose the next best execution plan. That means the optimizer might select an entirely different execution plan or, as above, a similar execution plan with another index. In this case it uses an index on SALE_DATE, which is a leftover from the previous chapter.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

From the optimizer's perspective, this index has two advantages over SALES_SUB_EUR. The optimizer believes that the filter on SALE_DATE is more selective than the one on SUBSIDIARY_ID. You can see that in the respective "Rows" column of the last two execution plans (about 10,000 versus 40,000). These estimations are, however, purely arbitrary because the query uses bind parameters. The SALE_DATE condition could, for example, select the entire table when providing the date of the first sale.

The second advantage of the SALES_DATE index is that is has a better clustering factor. This is a valid reason because the SALES table only grows chronologically. New rows are always appended to the end of the table as long as there are no rows deleted. The table order therefore corresponds to the index order because both are roughly sorted chronologically—the index has a good clustering factor.

When using an index with a good clustering factor, the selected tables rows are stored closely together so that the database only needs to read a few table blocks to get all the rows. Using this index, the query might be fast enough without an index-only scan. In this case we should remove the unneeded columns from the other index again.



Note

Some indexes have a good clustering factor automatically so that the performance advantage of an index-only scan is minimal.

In this particular example, there was a happy coincidence. The new filter on SALE_DATE not only prevented an index-only scan but also opened a new access path at the same time. The optimizer was therefore able to limit the performance impact of this change. It is, however, also possible to prevent an index only scan by adding columns to other clauses. However adding a column to the **`select`** clause can never open a new access path which could limit the impact of losing the index-only scan.



Tip

Maintain your index-only scans.

Add comments that remind you about an index-only scan and refer to that page so anyone can read about it.

Function-based indexes can also cause unpleasant surprises in connection with index-only scans. An index on `UPPER(last_name)` cannot be used for an index-only scan when selecting the `LAST_NAME` column. In the previous section we should have indexed the `LAST_NAME` column itself to support the `LIKE` filter and allow it to be used for an index-only scan when selecting the `LAST_NAME` column.



Tip

Always aim to index the original data as that is often the most useful information you can put into an index.

Avoid function-based indexing for expressions that cannot be used as access predicates.

Aggregating queries like the one shown above make good candidates for index-only scans. They query many rows but only a few columns, making a slim index sufficient for supporting an index-only scan. The more columns you query, the more columns you have to add to the indexed to support an index-only scan. As a developer you should therefore only select the columns you really need.



Tip

Avoid **`select *`** and fetch only the columns you need.

Regardless of the fact that indexing many rows needs a lot of space, you can also reach the limits of your database. Most databases impose rather rigid limits on the number of columns per index and the total size of an index entry. That means you cannot index an arbitrary number of columns nor arbitrarily long columns. The following overview lists the most important limitations. Nevertheless there are indexes that cover an entire table as we see in the next section.



Think about it

Queries that do not select any table columns are often executed with index-only scans.

Can you think of a meaningful example?

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

MySQL

MySQL 5.6 with InnoDB limits every single column to 767 bytes and all columns together to 3072 bytes. MyISAM indexes are limited to 16 columns and a maximum key length of 1000 bytes.

MySQL has a unique feature called "prefix indexing" (sometimes also called "partial indexing"). This means indexing only the first few characters of a column—so it has nothing to do with the partial indexes described in Chapter 2. If you index a column that exceeds the allowed column length (767 bytes for InnoDB), MySQL automatically truncates the column accordingly. This is the reason the **`create index`** statement succeeds with the warning "Specified key was too long; max key length is 767 bytes" if you exceed the limit. That means that the index doesn't contain a full copy of the column anymore and is therefore of limited use for an index-only scan (similar to a function-based index).

You can use MySQL's prefix indexing explicitly to prevent exceeding the total key length limit if you get the error message "Specified key was too long; max key length is [1000/3072] bytes." The following example only indexes the first ten characters of the `LAST_NAME` column.

```
CREATE INDEX .. ON employees (last_name(10));
```

Oracle Database

The maximum index key length depends on the block size and the index storage parameters (75% of the database block size minus some overhead). A B-tree index is limited to 32 columns.

When using Oracle 11*g* with all defaults in place (8k blocks), the maximum index key length is 6398 bytes. Exceeding this limit causes the error message "ORA-01450: maximum key length (6398) exceeded."

PostgreSQL

The PostgreSQL database supports index-only scans since release 9.2.

The key length of B-tree indexes is limited to 2713 bytes (hardcoded, approx. `BLCKSZ/3`). The respective error message "*index row size … exceeds btree maximum, 2713*" appears only when executing an **`insert`** or **`update`** that exceeds the limit. B-tree indexes can contain up to 32 columns.

SQL Server
: SQL Server limits the key length to 900 bytes and 16 key columns. Nevertheless, SQL Server has a feature that allows you to add arbitrarily long columns to an index for the sole purpose of supporting an index-only scan. For that, SQL Server distinguishes between key columns and nonkey columns.

  Key columns are index columns as they were discussed so far. Nonkey columns are additional columns that are only stored in the index leaf nodes. Nonkey columns can be arbitrarily long but cannot be used as access predicates (seek predicates).

  Nonkey columns are defined with the **`include`** keyword of the **`create index`** command:

```
 CREATE INDEX empsubupnam
     ON employees
       (subsidiary_id, last_name)
INCLUDE(phone_number, first_name);
```

Index-Organized Tables

The index-only scan executes an SQL statement using only the redundant data stored in the index. The original data in the heap table is not needed. If we take that concept to the next level and put all columns into the index, you may wonder why we need the heap table.

Some databases can indeed use an index as primary table store. The Oracle database calls this concept *index-organized tables (IOT)*, other databases use the term *clustered index*. In this section, both terms are used to either put the emphasis on the table or the index characteristics as needed.

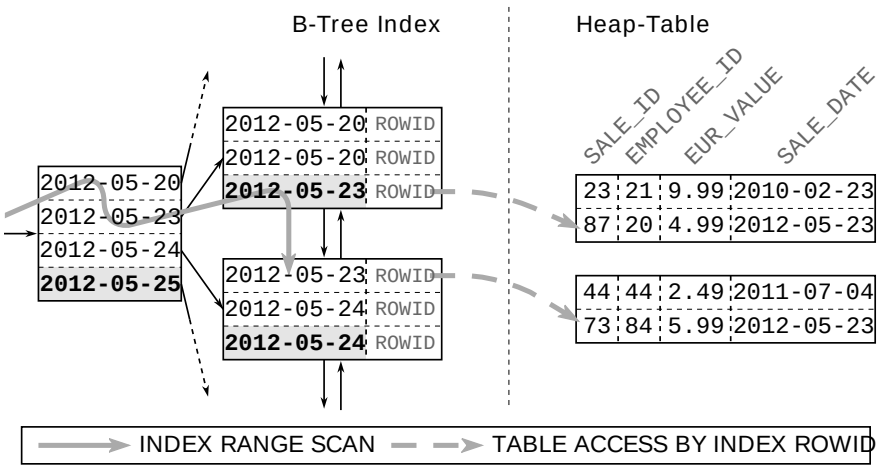
An index-organized table is thus a B-tree index without a heap table. This results in two benefits: (1) it saves the space for the heap structure; (2) every access on a clustered index is automatically an index-only scan. Both benefits sound promising but are hardly achievable in practice.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

The drawbacks of an index-organized table become apparent when creating another index on the same table. Analogous to a regular index, a so-called *secondary index* refers to the original table data—which is stored in the clustered index. There, the data is not stored statically as in a heap table but can move at any time to maintain the index order. It is therefore not possible to store the physical location of the rows in the index-organized table in the secondary index. The database must use a logical key instead.

The following figures show an index lookup for finding all sales on May 23rd 2012. For comparison, we will first look at Figure 5.1 that shows the process when using a heap table. The execution involves two steps: (1) the `INDEX RANGE SCAN`; (2) the `TABLE ACCESS BY INDEX ROWID`.

Figure 5.1. Index-Based Access on a Heap Table



Although the table access might become a bottleneck, it is still limited to one read operation per row because the index has the `ROWID` as a direct pointer to the table row. The database can immediately load the row from the heap table because the index has its exact position. The picture changes, however, when using a secondary index on an index-organized table. A secondary index does not store a physical pointer (`ROWID`) but only the key values of the clustered index—the so-called *clustering key*. Often that is the primary key of the index-organized table.

Accessing a secondary index does not deliver a ROWID but a logical key for searching the clustered index. A single access, however, is not sufficient for searching clustered index—it requires a full tree traversal. That means that accessing a table via a secondary index searches two indexes: the secondary index once (INDEX RANGE SCAN), then the clustered index *for each row* found in the secondary index (INDEX UNIQUE SCAN).

Figure 5.2. Secondary Index on an IOT

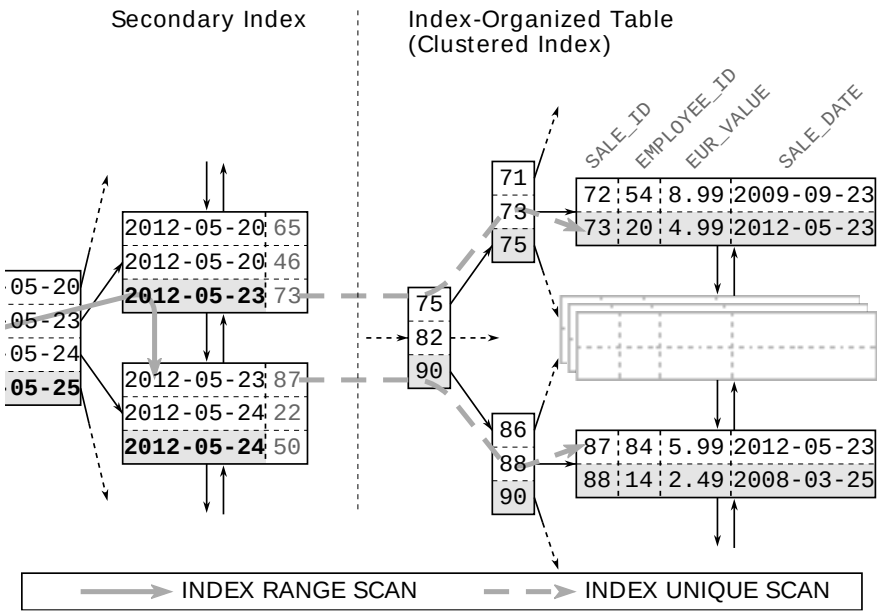


Figure 5.2 makes it clear, that the B-tree of the clustered index stands between the secondary index and the table data.

Accessing an index-organized table via a secondary index is very inefficient, and it can be prevented in the same way one prevents a table access on a heap table: by using an index-only scan—in this case better described as "secondary-index-only scan". The performance advantage of an index-only scan is even bigger because it not only prevents a single access but an entire INDEX UNIQUE SCAN.



Important

Accessing an index-organized table via a secondary index is very inefficient.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Using this example we can also see that databases exploit all the redundancies they have. Bear in mind that a secondary index stores the clustering key for each index entry. Consequently, we can query the clustering key from a secondary index without accessing the index-organized table:

```
SELECT sale_id
  FROM sales_iot
 WHERE sale_date = ?;

-------------------------------------------------
| Id | Operation        | Name           | Cost |
-------------------------------------------------
|  0 | SELECT STATEMENT |                |    4 |
|* 1 |  INDEX RANGE SCAN| SALES_IOT_DATE |    4 |
-------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - access("SALE_DATE"=:DT)
```

The table `SALES_IOT` is an index-organized table that uses `SALE_ID` as clustering key. Although the index `SALE_IOT_DATE` is on the `SALE_DATE` column only, it still has a copy of the clustering key `SALE_ID` so it can satisfy the query using the secondary index only.

When selecting other columns, the database has to run an `INDEX UNIQUE SCAN` on the clustered index for each row:

```
SELECT eur_value
  FROM sales_iot
 WHERE sale_date = ?;

-------------------------------------------------
| Id  | Operation         | Name           | Cost |
-------------------------------------------------
|   0 | SELECT STATEMENT  |                |   13 |
|*  1 |  INDEX UNIQUE SCAN| SALES_IOT_PK   |   13 |
|*  2 |   INDEX RANGE SCAN| SALES_IOT_DATE |    4 |
-------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - access("SALE_DATE"=:DT)
   2 - access("SALE_DATE"=:DT)
```

Index-organized tables and clustered indexes are, after all, not as useful as it seems at first sight. Performance improvements on the clustered index are easily lost on when using a secondary index. The clustering key is usually longer than a ROWID so that the secondary indexes are larger than they would be on a heap table, often eliminating the savings from the omission of the heap table. The strength of index-organized tables and clustered indexes is mostly limited to tables that do not need a second index. Heap tables have the benefit of providing a stationary master copy that can be easily referenced.



Important

Tables with one index only are best implemented as clustered indexes or index-organized tables.

Tables with more indexes can often benefit from heap tables. You can still use index-only scans to avoid the table access. This gives you the **`select`** performance of a clustered index without slowing down other indexes.

Database support for index-organized tables and clustered index is very inconsistent. The overview on the next page explains the most important specifics.

Why Secondary Indexes have no ROWID

A direct pointer to the table row would be desirable for a secondary index as well. But that is only possible, if the table row stays at fixed storage positions. That is, unfortunately, not possible if the row is part of an index structure, which is kept in order. Keeping the index order needs to move rows occasionally. This is also true for operations that do not affect the row itself. An **`insert`** statement, for example, might split a leaf node to gain space for the new entry. That means that some entries are moved to a new data block at a different place.

A heap table, on the other hand, doesn't keep the rows in any order. The database saves new entries wherever it finds enough space. Once written, data doesn't move in heap tables.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

MySQL
: The MyISAM engine only uses heap tables while the InnoDB engine always uses clustered indexes. That means you do not directly have a choice.

Oracle Database
: The Oracle database uses heap tables by default. Index-organized tables can be created using the `ORGANIZATION INDEX` clause:

```
CREATE TABLE (
   id    NUMBER NOT NULL PRIMARY KEY,
   [...]
) ORGANIZATION INDEX;
```

  The Oracle database always uses the primary key as the clustering key.

PostgreSQL
: PostgreSQL only uses heap tables.

  You can, however, use the `CLUSTER` clause to align the contents of the heap table with an index.

SQL Server
: By default SQL Server uses clustered indexes (index-organized tables) using the primary key as clustering key. Nevertheless you can use arbitrary columns for the clustering key—even non-unique columns.

  To create a heap table you must use the `NONCLUSTERED` clause in the primary key definition:

```
CREATE TABLE (
   id    NUMBER NOT NULL,
   [...]
   CONSTRAINT pk PRIMARY KEY NONCLUSTERED (id)
);
```

  Dropping a clustered index transforms the table into a heap table.

  SQL Server's default behavior often causes performance problems when using secondary indexes.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Chapter 6

Sorting and Grouping

Sorting is a very resource intensive operation. It needs a fair amount of CPU time, but the main problem is that the database must temporarily buffer the results. After all, a sort operation must read the complete input before it can produce the first output. Sort operations cannot be executed in a pipelined manner—this can become a problem for large data sets.

An index provides an ordered representation of the indexed data: this principle was already described in Chapter 1. We could also say that an index stores the data in a presorted fashion. The index is, in fact, sorted just like when using the index definition in an **order by** clause. It is therefore no surprise that we can use indexes to avoid the sort operation to satisfy an **order by** clause.

Ironically, an INDEX RANGE SCAN also becomes inefficient for large data sets—especially when followed by a table access. This can nullify the savings from avoiding the sort operation. A FULL TABLE SCAN with an explicit sort operation might be even faster in this case. Again, it is the optimizer's job to evaluate the different execution plans and select the best one.

An indexed **order by** execution not only saves the sorting effort, however; it is also able to return the first results without processing all input data. The **order by** is thus executed in a *pipelined* manner. Chapter 7, "Partial Results", explains how to exploit the pipelined execution to implement efficient pagination queries. This makes the pipelined **order by** so important that I refer to it as the *third power of indexing*.

This chapter explains how to use an index for a pipelined **order by** execution. To this end we have to pay special attention to the interactions with the **where** clause and also to ASC and DESC modifiers. The chapter concludes by applying these techniques to **group by** clauses as well.

Indexing Order By

SQL queries with an **order by** clause do not need to sort the result explicitly if the relevant index already delivers the rows in the required order. That means the same index that is used for the **where** clause must also cover the **order by** clause.

As an example, consider the following query that selects yesterday's sales ordered by sale data and product ID:

```
SELECT sale_date, product_id, quantity
  FROM sales
 WHERE sale_date = TRUNC(sysdate) - INTERVAL '1' DAY
 ORDER BY sale_date, product_id;
```

There is already an index on SALE_DATE that can be used for the **where** clause. The database must, however, perform an explicit sort operation to satisfy the **order by** clause:

```
-------------------------------------------------------------------
|Id | Operation                     | Name       | Rows | Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT              |            |  320 |   18 |
| 1 |  SORT ORDER BY                |            |  320 |   18 |
| 2 |   TABLE ACCESS BY INDEX ROWID| SALES      |  320 |   17 |
|*3 |    INDEX RANGE SCAN           | SALES_DATE |  320 |    3 |
-------------------------------------------------------------------
```

An INDEX RANGE SCAN delivers the result in index order anyway. To take advantage of this fact, we just have to extend the index definition so it corresponds to the **order by** clause:

```
  DROP INDEX sales_date;
CREATE INDEX sales_dt_pr ON sales (sale_date, product_id);
```

```
-------------------------------------------------------------------
|Id | Operation                     | Name        | Rows | Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT              |             |  320 |  300 |
| 1 |  TABLE ACCESS BY INDEX ROWID| SALES       |  320 |  300 |
|*2 |   INDEX RANGE SCAN            | SALES_DT_PR |  320 |    4 |
-------------------------------------------------------------------
```


Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

The sort operation SORT ORDER BY disappeared from the execution plan even though the query still has an **order by** clause. The database exploits the index order and skips the explicit sort operation.



Important

If the index order corresponds to the **order by** clause, the database can omit the explicit sort operation.

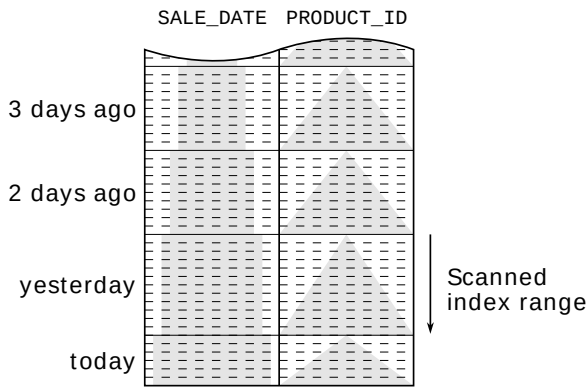
Even though the new execution plan has fewer operations, the cost value has increased considerably because the clustering factor of the new index is worse (see "Automatically Optimized Clustering Factor" on page 133). At this point, it should just be noted that the cost value is not always a good indicator of the execution effort.

For this optimization, it is sufficient that the scanned index range is sorted according to the **order by** clause. Thus the optimization also works for this particular example when sorting by PRODUCT_ID only:

```
SELECT sale_date, product_id, quantity
  FROM sales
 WHERE sale_date = TRUNC(sysdate) - INTERVAL '1' DAY
 ORDER BY product_id;
```

In Figure 6.1 we can see that the PRODUCT_ID is the only relevant sort criterion in the scanned index range. Hence the index order corresponds to the **order by** clause in *this index range* so that the database can omit the sort operation.

Figure 6.1. Sort Order in the Relevant Index Range



This optimization can cause unexpected behavior when extending the scanned index range:

```
SELECT sale_date, product_id, quantity
  FROM sales
 WHERE sale_date >= TRUNC(sysdate) - INTERVAL '1' DAY
 ORDER BY product_id;
```

This query does not retrieve *yesterday's* sales but all sales *since yesterday*. That means it covers several days and scans an index range that is not exclusively sorted by the PRODUCT_ID. If we look at Figure 6.1 again and extend the scanned index range to the bottom, we can see that there are again smaller PRODUCT_ID values. The database must therefore use an explicit sort operation to satisfy the **order by** clause.

```
-------------------------------------------------------------------
|Id |Operation                     | Name        | Rows | Cost |
-------------------------------------------------------------------
| 0 |SELECT STATEMENT              |             |  320 |  301 |
| 1 | SORT ORDER BY                |             |  320 |  301 |
| 2 |  TABLE ACCESS BY INDEX ROWID| SALES       |  320 |  300 |
|*3 |   INDEX RANGE SCAN           | SALES_DT_PR |  320 |    4 |
-------------------------------------------------------------------
```

If the database uses a sort operation even though you expected a pipelined execution, it can have two reasons: (1) the execution plan with the explicit sort operation has a better cost value; (2) the index order in the scanned index range does not correspond to the **order by** clause.

A simple way to tell the two cases apart is to use the full index definition in the **order by** clause—that means adjusting the query to the index in order to eliminate the second cause. If the database still uses an explicit sort operation, the optimizer prefers this plan due to its cost value; otherwise the database cannot use the index for the original **order by** clause.



Tip

Use the full index definition in the **order by** clause to find the reason for an explicit sort operation.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

In both cases, you might wonder if and how you could possibly reach a pipelined **order by** execution. For this you can execute the query with the full index definition in the **order by** clause and inspect the result. You will often realize that you have a false perception of the index and that the index order is indeed not as required by the original **order by** clause so the database cannot use the index to avoid a sort operation.

If the optimizer prefers an explicit sort operation for its cost value, it is usually because the optimizer takes the best execution plan for the *full execution* of the query. In other words, the optimizer opts for the execution plan which is the fastest to get the last record. If the database detects that the application fetches only the first few rows, it might in turn prefer an indexed **order by**. Chapter 7, "Partial Results", explains the corresponding optimization methods.

Automatically Optimized Clustering Factor

The Oracle database keeps the clustering factor at a minimum by considering the ROWID for the index order. Whenever two index entries have the same key values, the ROWID decides upon their final order. The index is therefore also ordered according to the table order and thus has the smallest possible clustering factor because the ROWID represents the physical address of table row.

By adding another column to an index, you insert a new sort criterion *before* the ROWID. The database has less freedom in aligning the index entries according to the table order so the index clustering factor can only get worse.

Regardless, it is still possible that the index order roughly corresponds to the table order. The sales of a day are probably still clustered together in the table as well as in the index—even though their sequence is not exactly the same anymore. The database has to read the table blocks multiple times when using the SALE_DT_PR index—but these are just the same table blocks as before. Due to the caching of frequently accessed data, the performance impact could be considerably lower than indicated by the cost values.

Indexing ASC, DESC and NULLS FIRST/LAST

Databases can read indexes in both directions. That means that a pipelined **order by** is also possible if the scanned index range is in the exact opposite order as specified by the **order by** clause. Although ASC and DESC modifiers in the **order by** clause can prevent a pipelined execution, most databases offer a simple way to change the index order so an index becomes usable for a pipelined **order by**.

The following example uses an index in reverse order. It delivers the sales since yesterday ordered by descending date and descending PRODUCT_ID.

```
SELECT sale_date, product_id, quantity
  FROM sales
 WHERE sale_date >= TRUNC(sysdate) - INTERVAL '1' DAY
 ORDER BY sale_date DESC, product_id DESC;
```

The execution plan shows that the database reads the index in a descending direction.

```
-------------------------------------------------------------------
|Id |Operation                     | Name        | Rows | Cost |
-------------------------------------------------------------------
| 0 |SELECT STATEMENT              |             |  320 |  300 |
| 1 | TABLE ACCESS BY INDEX ROWID | SALES       |  320 |  300 |
|*2 |  INDEX RANGE SCAN DESCENDING| SALES_DT_PR |  320 |    4 |
-------------------------------------------------------------------
```

In this case, the database uses the index tree to find the *last* matching entry. From there on, it follows the leaf node chain "upwards" as shown in Figure 6.2. After all, this is why the database uses a *doubly* linked list to build the leaf node chain.

Of course it is crucial that the scanned index range is in the exact opposite order as needed for the **order by** clause.



Important

Databases can read indexes in both directions.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>